

Trajectory Experiment Analysis

Overview

The trajectory experiment evaluates whether incorporating a multi-step query history (`trajectory_depth > 1`) improves cache retrieval quality. For each **key builder** $\times$ **weight** combination chosen from Phase 1, the same configuration is re-run at **trajectory depths 3, 4, 5, 6** on `libero_spatial` (10 tasks, 5 episodes each).

The Phase 1 results (single-step lookup, equivalent to `trajectory_depth = 1`) are merged into the analysis as the **depth-1 baseline** for direct comparison.

Configuration Selection

Per key builder, the trajectory experiment uses **3 weight configurations** chosen from Phase 1 rankings:

- the **top-1** weight (best Phase 1 result),
- the **top-2** weight (second-best Phase 1 result),
- the **second-worst** weight (deliberately included to test whether trajectory aggregation can rescue weak baselines).

This gives **5 key builders** $\times$ **3 weights** $\times$ **4 depths** = **60 trajectory runs**, plus the 15 corresponding Phase 1 runs as the depth-1 baseline.

KB ID	Type	Selected weights (top1 / top2 / 2nd-worst)
a	<code>cp1_mean_pool</code>	w5 / w7 / w3
b1	<code>cp1_spatial_pool_16</code>	w8 / w4 / w3
b2	<code>cp1_spatial_pool_64</code>	w5 / w8 / w4
c	<code>cp1_max_pool</code>	w4 / w7 / w3
d	<code>clip</code>	w6 / w7 / w3

Weight definitions are identical to Phase 1 (`v0` / `v1` =vision_0/1, `rs` =robot_state; `prompt_emb` and `vision_2` are disabled).

Results

Full Results Table (Success Rate, %)

Key Builder	Weight	Phase1 (d=1)	d=3	d=4	d=5	d=6	Best Depth	Δ vs Phase1
mean_pool	w3 (2nd-worst)	42	52	52	56	58	6	+16
mean_pool	w5 (top-1)	58	60	58	56	56	3	+2
mean_pool	w7 (top-2)	56	64	58	56	52	3	+8
spatial_16	w3 (2nd-worst)	44	50	54	60	60	5	+16
spatial_16	w4 (top-2)	58	56	50	52	52	3	-2
spatial_16	w8 (top-1)	62	60	68	66	62	4	+6
spatial_64	w4 (2nd-worst)	40	50	50	52	52	5	+12
spatial_64	w5 (top-1)	64	58	54	58	58	3	-6
spatial_64	w8 (top-2)	60	50	62	64	62	5	+4
max_pool	w3 (2nd-worst)	42	50	54	60	54	5	+18
max_pool	w4 (top-1)	56	50	46	52	50	5	-4
max_pool	w7 (top-2)	54	54	52	54	56	6	+2
clip	w3 (2nd-worst)	44	62	66	66	64	4	+22

Key Builder	Weight	Phase1 (d=1)	d=3	d=4	d=5	d=6	Best Depth	Δ vs Phase1
clip	w6 (top-1)	66	62	60	66	60	5	+0
clip	w7 (top-2)	66	58	68	66	68	4	+2

Per-Depth Aggregate (mean over all 15 configurations)

Depth	Mean	Min	Max	n
1 (Phase1)	54.1%	40%	66%	15
3	55.7%	50%	64%	15
4	56.8%	46%	68%	15
5	58.9%	52%	66%	15
6	57.6%	50%	68%	15

Per-Key-Builder Aggregate (depths 3-6)

Rank	Key Builder	Mean	Min	Max
1	clip	63.8%	58%	68%
2	spatial_16	57.5%	50%	68%
3	mean_pool	56.5%	52%	64%
4	spatial_64	55.8%	50%	64%
5	max_pool	52.7%	46%	60%

Analysis

1. Trajectory Aggregation Has a Small Average Effect

The mean success rate climbs from **54.1% at depth 1** to **58.9% at depth 5**, a gain of just under five percentage points, then drops slightly at depth 6. This is the headline result: trajectory aggregation does help, **but not by a large margin in the average case**, and **deeper is not always better**. The peak at depth 5 with a regression at

depth 6 suggests that further history introduces stale or noisy queries that dilute the relevance of the most recent step.

2. Trajectory Rescues Weak Baselines, But Barely Moves Strong Ones

Splitting the 15 configurations by their Phase 1 role (top-1 / top-2 / second-worst) makes the effect much sharper than the per-depth average suggests:

Phase1 Role	n	Mean Δ (best traj depth – Phase1)
2nd-worst	5	+16.8 pp
top-2	5	+3.6 pp
top-1	5	+1.6 pp

Every single second-worst configuration gains double-digit success rate when trajectory is enabled — `clip/w3` jumps from 44% to 66% (+22 pp), `max_pool/w3` from 42% to 60% (+18 pp). In contrast, **strong Phase 1 configurations typically gain almost nothing** and several actively regress (`spatial_64/w5` : –6 pp, `max_pool/w4` : –4 pp, `spatial_16/w4` : –2 pp).

The interpretation is straightforward: when the single-step query is already a good index into the cache, mixing in older queries can only add noise. Trajectory aggregation pays off precisely when the per-step features are weak — it smooths out single-step retrieval errors at the cost of temporal specificity. **Trajectory is a robustness mechanism, not a capability boost.**

3. Key Builder Choice Still Dominates

Even with trajectory enabled, `clip` is the clear winner, averaging 63.8% across depths 3–6 — six full points ahead of the next best (`spatial_16` at 57.5%) and over eleven points ahead of `max_pool`. Four of the top-five single-run results in the entire experiment use `clip`, with the global maximum of **68%** reached by `clip/w7` at depths 4 and 6 and by `spatial_16/w8` at depth 4.

The lesson reinforces the Phase 1 conclusion: **upgrading the key representation gives larger returns than tuning trajectory depth or weights**. Trajectory is best understood as a complementary lever, not a replacement for a better feature extractor.

4. Optimal Depth Lies in the Middle

For configurations that benefit from trajectory at all, the best depth is almost always **4 or 5**:

- 7 of 15 configurations peak at depth 5,
- 4 peak at depth 4,
- 3 peak at depth 3,
- only 1 peaks at depth 6 (and that one — `mean_pool/w3` — peaks at 58% by a single point over depth 5).

Combined with the per-depth aggregate showing depth 6 falling back below depth 5, this argues against searching deeper than 5 in future experiments.

5. Strong-Weight Baselines Become Less Stable Under Trajectory

A subtle but important effect: looking at the variance of strong baselines across depths, `clip/w6`, `spatial_16/w4`, `max_pool/w4`, and `spatial_64/w5` all show non-monotonic and sometimes lower scores under trajectory than at depth 1. This is consistent with the rescue/dilution interpretation in §2 — trajectory **replaces single-step variance with temporal-mixing variance**, which is not necessarily smaller.

If trajectory is enabled in production, the weight configuration should be re-tuned for the trajectory regime; importing the Phase 1 optimum directly is not safe.

Recommendations for Phase 3

1. **Default to `clip` as the key builder.** It dominates the success-rate ranking with or without trajectory, and the trajectory peak (`clip/w7` at $d=4$ or $d=6$, 68%) is the global best of the experiment. The ~3 GB VRAM overhead noted in Phase 1 remains the only reason not to.
2. **Use trajectory depth 4 or 5, not 6.** Depth 6 underperforms depth 5 on the aggregate and rarely peaks for any single configuration. Searching beyond depth 5 is unlikely to be productive.
3. **Re-tune weights specifically for the trajectory regime.** The Phase 1 ranking is not preserved under trajectory: several Phase 1 top-1 weights regress, and the equally-weighted `w3` configurations move from being the worst to being competitive. A focused weight sweep over the `(v0, v1, rs)` simplex at fixed depth 5 would have higher information value than another depth sweep.

4. **Treat trajectory as a robustness add-on, not a quality boost.** When the underlying single-step retrieval is already good, trajectory aggregation gives marginal or negative returns. It is most useful when the per-step query is noisy — for example, on a new task suite, with a weaker key builder, or under distribution shift. Future experiments should test it in exactly those settings.
 5. **Investigate adaptive depth.** All experiments here used a fixed depth. An adaptive scheme that down-weights older queries when the most recent query is high-confidence (or simply switches to depth=1) could plausibly capture the upside of trajectory on weak queries without paying the cost on strong ones.
-

Limitations

Sample sizes are identical to Phase 1 (5 episodes per task × 10 tasks = 50 trials per configuration). Differences of one or two percentage points are within sampling noise; only the larger effects discussed above (the +12 to +22 gains on weak baselines, the clip dominance, the depth-5 peak) are robust to this scale. As before, a final candidate configuration (e.g. `clip/w7` at depth 4 or 5) should be re-validated with a substantially larger episode count before being adopted as the production default.

One configuration (`traj_d5_043_d_rrf_w6`) crashed with a native segmentation fault during its first batch and was successfully re-run in isolation; the final result is included in the table above. No other runs required retries.